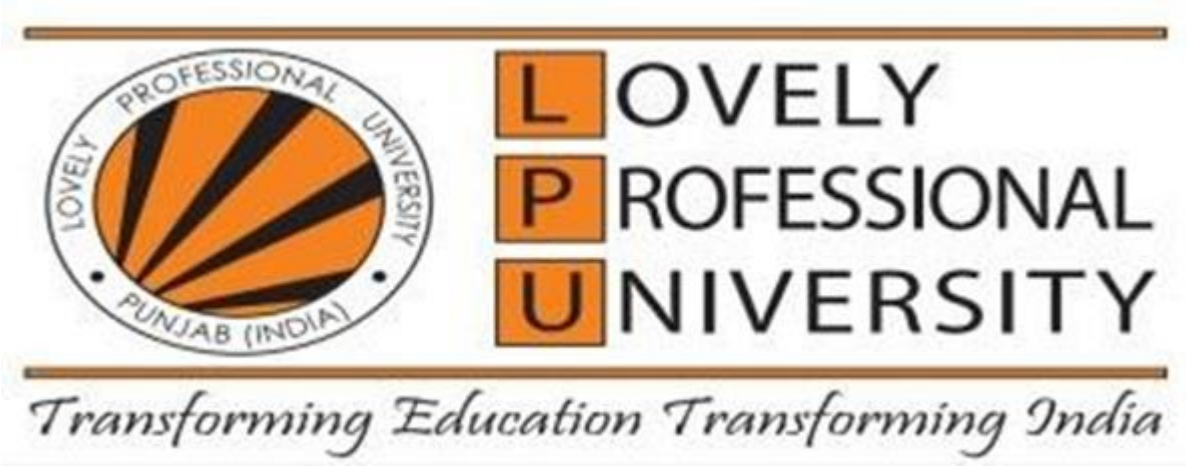

A PROJECT REPORT

on

LogSentinel: A Simple Log Analysis & Anomaly Detection Website

Submitted by

[Anopa Tatenda Chagonda] — Roll No. [27]

[Pamela B Mbatha] — Roll No. [12]

[Nkosinodumo Onalena Ngwenya] — Roll No. [23]

[Divine Anotida Mkusa] — Roll No. [09]

Under the Supervision of

[Manish Singh]

[Network Security and Ethical Hacking]

Submitted to

[Lovely Professional University]

In partial fulfillment of the requirement for the award of

[Bachelor of Computer Applications]

Batch 2024–2027

CERTIFICATE FROM SUPERVISOR

Date: [DD/MM/YYYY]

This is to certify that the project report titled “LogSentinel: A Simple Log Analysis & Anomaly Detection Website” submitted by the group members named on the Team Members page, was carried out under my supervision, and is the group's own work.

Signature: _____

(**Manish Singh**, ……, LPU)

STUDENT'S DECLARATION

We, the undersigned team members, declare that this project report is our own group work, done under the guidance of [**Manish Singh**], and has not been copied from any other source or submitted anywhere else before.

Date: _07/07/2026_____

Place: _LPU_____

Signatures:

1. _____anopa_____ [Anopa Tatenda Chagonda], R**A**oll No. [27]

2. _____Pamela_____ [Pamela B Mbatha], Roll No. [12]

3. _____Onalena_____ [Nkosinodumo Onalena Ngwenya], Roll No. [23]

4. _____Divine_____ [Divine Anotida Mkusa], Roll No. [09]

TEAM MEMBERS AND CONTRIBUTION

This project was completed as a team of four members. Table A lists each member and their primary area of contribution; in practice, all members were involved in discussion, testing, and review of the whole project.

Roll No.	Name	Primary Contribution
[27]	[Anopa Tatenda Chagonda]	Log parser (parser.py) and log format research
[12]	[Pamela Bonolo Mbatha]	Anomaly detection logic (detector.py) and thresholds
[23]	[Nkosinodumo Onalena Ngwenya]	Database design (db.py) and Flask API (app.py)
[09]	[Divine Anotida Mkusa]	Dashboard/frontend (HTML, CSS, Chart.js) and testing

Table A — Team Members and Contribution

ACKNOWLEDGEMENTS

We would like to thank our project guide, [Name of the Guide], for helping and guiding us throughout this project. We are also grateful to [University/Institute Name] for giving us the opportunity and resources to complete this work. Lastly, we thank our families and friends for their support.

Signatures:

1. _____Anopa_____ 2. _____Pamela_____ 3. _____Onalena_____
4. _____Divine_____

CONTENT

LIST OF ILLUSTRATIONS

ABSTRACT

Websites and servers keep a record of everything that happens on them — every visit, every login attempt, every error. This record is called a log file. In most small setups, nobody actually looks at these files, so warning signs of an attack (like someone trying many passwords, or a bot scanning for hidden pages) go unnoticed.

This project, Log Sentinel, is a simple website built with Python and Flask that reads log files, understands what's in them, stores the information in a small database, and automatically looks for five kinds of suspicious behavior: repeated failed logins, too many “page not found” errors from one visitor, too many requests too fast, visitors using known hacking-tool software, and access happening at odd hours. The results are shown on a dashboard with simple charts so anyone can understand what's going on without reading raw log text.

The project was tested on a sample log file containing one clear example of each type of suspicious activity. All five types were correctly found and labeled by how serious they were. This shows that even a small, simple project can catch the most common warning signs that a real attacker leaves behind.

CHAPTER I: INTRODUCTION

1.1 Why This Project

Every website and server keeps writing down what happens on it — who visited, when, and what they asked for. These records pile up quickly and are almost never read by a human unless something has already gone wrong. Big companies use expensive security software to watch these logs automatically, but small websites, college labs, and individual projects usually can't afford that. we wanted to build something simple that does the basic version of that job — read the logs and flag the things worth noticing.

1.2 What the Project Does

Log Sentinel is a small website (built using Flask, a Python web framework) that a person can use to upload a log file. The project then:

7. Reads the file and understands each line using pattern matching (regex).
8. Saves the understood data into a small database (SQLite).
9. Checks the data for signs of five common problems.
10. Shows everything on a dashboard with charts, so it's easy to look at.

1.3 Project Summary

Item	Details
Title	Log Analysis & Anomaly Detection Website (“LogSentinel”)
Tools Used	Python, Flask, Pandas, Regex, SQLite, Chart.js
What It Does	Reads server log files, finds suspicious activity, and shows it on a dashboard.
Main Features	1. Finds repeated login attempts and scanning behaviour. 2. Shows charts of log activity. 3. Lists out anomalies with a seriousness level.

1.4 What We Read Before Starting

Before building this, We looked at how professional tools handle log files. The SANS Institute (a well-known security organization) recommends collecting logs in one place and checking them for both known attack signatures and unusual patterns of activity. We also read the standard documentation for how Apache and Nginx servers write their log lines, since my program had to be able to read those formats correctly. This gave me an idea of what a “real” version of this kind of tool looks for, even though my project is a much smaller, simpler version.

CHAPTER II: OBJECTIVES OF THE STUDY

The main goal of this project is to build a simple tool that can read raw log files and point out suspicious activity automatically, without needing expensive security software. To reach that goal, we broke it down into smaller, clear objectives:

11. Build a program that can read and understand log lines from web servers (Apache/Nginx) and from Linux login logs (syslog).
12. Store the information from the logs in a simple database so it can be searched and reused.
13. Write logic that checks the stored data for five common suspicious patterns: repeated failed logins, too many 404 errors, too many requests in general, suspicious visitor software, and access at odd hours.
14. Give each finding a seriousness level (low, medium, high, critical) so the most important ones stand out.
15. Build a dashboard with charts so the results are easy to understand at a glance.
16. Test the whole system on a sample log file to check that it actually catches what it's supposed to catch.

CHAPTER III: PROBLEM STATEMENT

Small websites and personal servers produce log files all the time, but almost nobody checks them regularly. This means warning signs of an attack — like someone trying many passwords in a row, or a bot searching for hidden admin pages — are usually only discovered after real damage has already happened.

The problem this project tries to solve is: given a raw log file, how can a simple program automatically read it, notice the patterns that usually mean trouble, and show them in a way a normal person (not a security expert) can understand and act on?

To solve this, we had to figure out three smaller things: how to read different log formats correctly using regex, how to decide what counts as “too many” events in a short time (so it doesn't cry wolf over normal traffic), and how to display the results in a simple, clear dashboard.

CHAPTER IV: METHODOLOGY

4.1 How We Approached the Project

This is a build-and-test kind of project rather than a survey-based one. As a team, we first decided what kinds of suspicious activity we wanted to catch, based on what security guides usually recommend watching for. We then divided the work and designed the project in separate parts, so each part could be built and tested on its own by different members. Finally, we tested the whole thing together using a sample log file to check that it actually works.

4.2 How the Project Is Structured

The project is split into four simple parts, shown in Figure 1: a part that reads the uploaded log file (parser), a part that saves the data (database), a part that checks for suspicious patterns (detector), and a part that shows everything as a dashboard (the website itself). Keeping these separate meant different team members could work on and test the log-reading part without worrying about the dashboard, and test the detection logic without worrying about how the file was uploaded.

Figure 4.1 — LogSentinel System Architecture

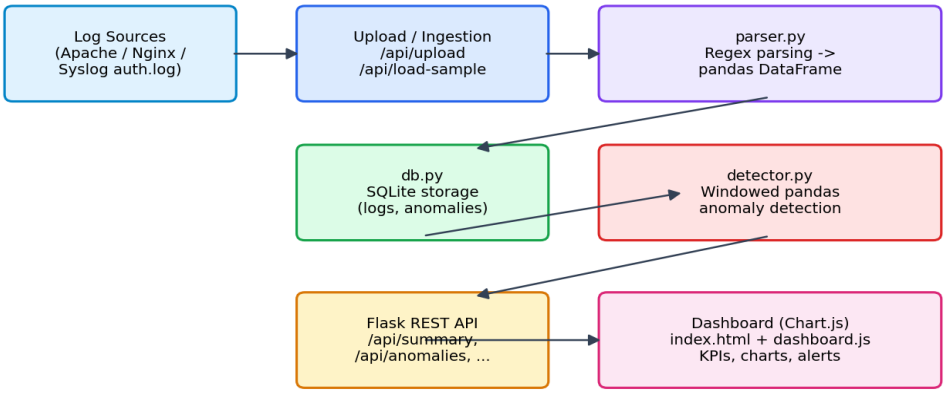

Figure 1 — How the Project Works (System Diagram)

4.3 Tools Used

Part of Project	Tool Used
Website / Backend	Python, Flask
Reading Log Files	Regex (pattern matching)
Processing Data	Pandas (a Python library for working with tables of data)
Storing Data	SQLite (a simple, file-based database)
Dashboard / Charts	HTML, CSS, JavaScript, Chart.js

Table 2 — Tools Used

4.4 Where the Test Data Came From

Instead of collecting real logs from a live website (which would be messy and wouldn't have a "correct answer" to check against), we used a sample log file that was built to contain exactly one clear example of each type of suspicious activity, mixed in with normal, everyday traffic. This let us test whether the detection logic actually worked, because we knew in advance what it was supposed to find.

4.5 How the Log Reading Works

Web server logs and login logs look different, so the program checks a file against a few different patterns to figure out its format automatically (it looks at the first few lines to decide). Once it knows the format, it goes line by line and pulls out the useful information: who made the request, when, what they asked for, and whether it succeeded or failed.

4.6 How the Detection Works

The basic idea behind almost every check is the same: look at one visitor (one IP address) and count how many times something happened within a short window of time. If that count crosses a limit, it gets flagged. Table 3 lists the five checks.

Check	Limit Used	What It Means
Repeated failed logins	5 in 5 minutes	Someone trying to guess a password
Too many 404 errors	10 in 1 minute	Someone searching for hidden pages/files
Too many requests overall	100 in 1 minute	Possible overload/attack attempt
Suspicious visitor software	Name match	Known scanning tools like sqlmap, Nmap
Access at odd hours	Outside 08:00–20:00	Activity when no one should normally be active

Table 3 — Detection Checks

Each flagged event is also given a seriousness level based on how far past the limit it went — far past the limit means “critical”, just barely past means “low”. This helps whoever is looking at the dashboard know what to check first.

CHAPTER V: DATA ANALYSIS AND INTERPRETATION

We tested the project using the sample log file, which had 803 website requests and 23 login-related log lines. All of these were read successfully — none were skipped or failed to be understood.

5.1 What Kind of Responses Showed Up

Figure 2 shows how many requests got each type of response code. Most were normal (success codes), but there's a clear group of failed-login (401) and page-not-found (404) responses — these line up with the suspicious activity that was built into the sample file.

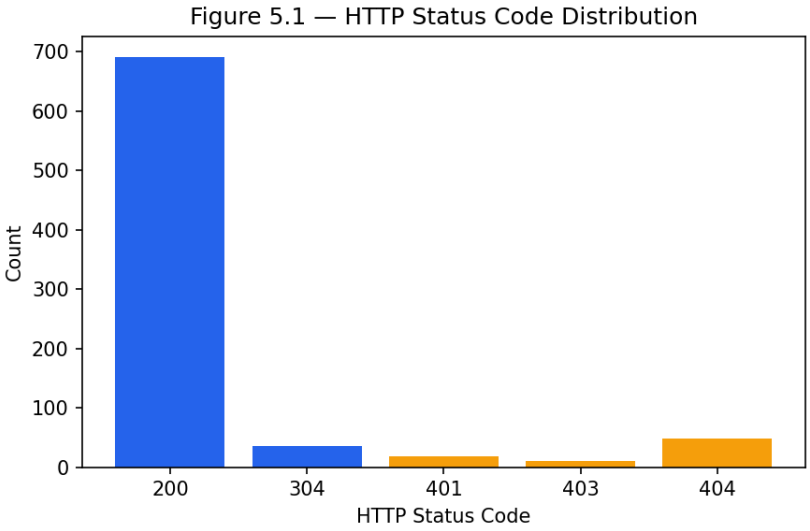

Figure 2 — Types of HTTP Responses in the Logs

5.2 What Time of Day Things Happened

Figure 3 shows activity by hour. The shaded areas mark “outside business hours” (before 8am, after 8pm). A small amount of activity in that shaded zone came from a repeat visitor, which is exactly the kind of thing the after-hours check is meant to catch.

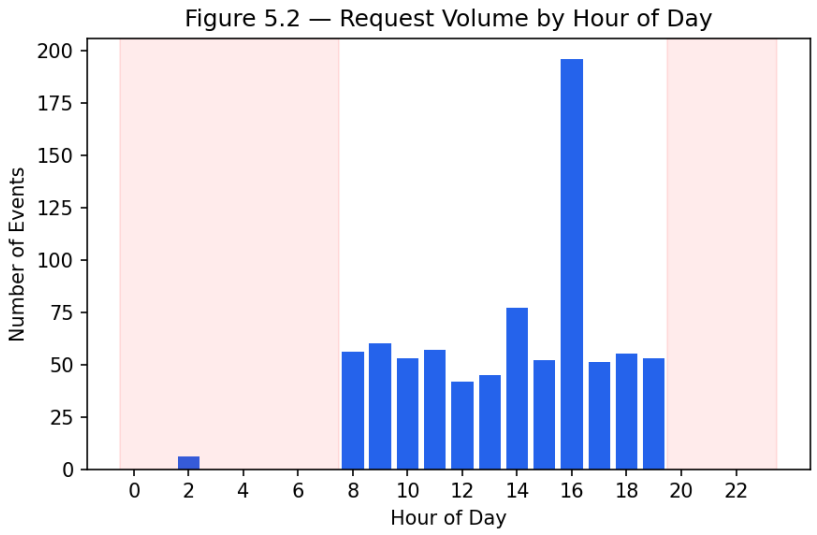

Figure 3 — What Time of Day Requests Happen

5.3 Which Visitors Sent the Most Traffic

Figure 4 shows the ten IP addresses responsible for the most requests. One IP address stands out with unusually high traffic — this turned out to be the same one flagged for sending too many requests too quickly.

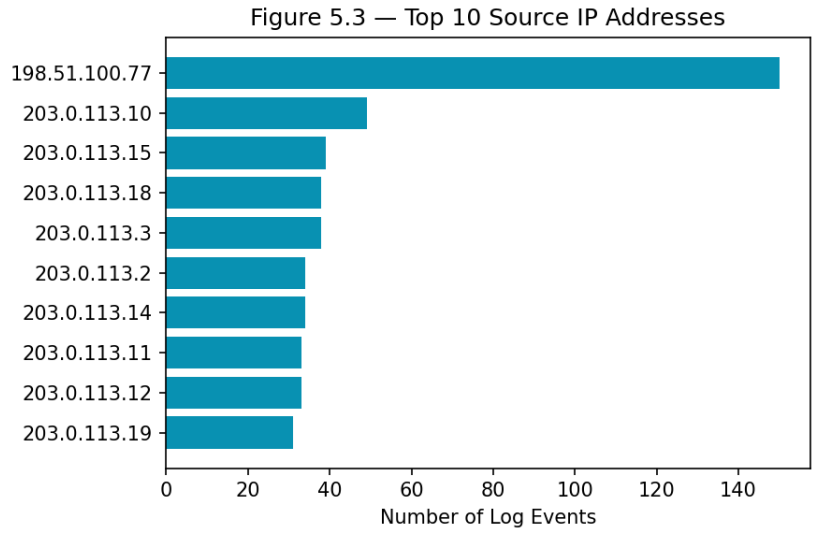

Figure 4 — Which IP Addresses Sent the Most Requests

5.4 Requests and Errors Over Time

Figure 6 plots total requests and error responses over time, in short time chunks. There are a couple of sharp spikes — these match up with the repeated-login and error-flood events, and one spike in total requests (without more errors) matches the “too many requests” case.

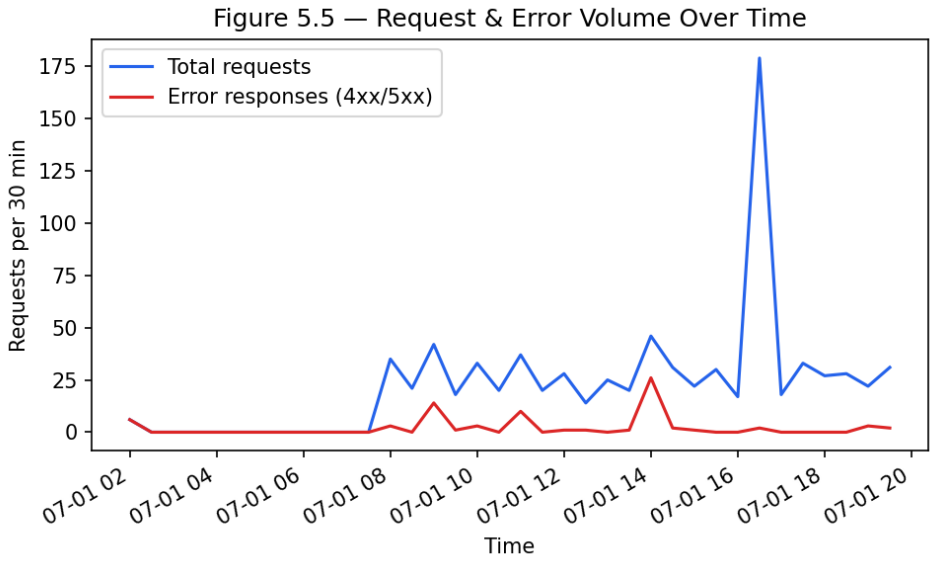

Figure 6 — Requests and Errors Over Time

5.5 What Was Actually Flagged

Running all five checks on the sample data produced 15 flagged items in total. Figure 5 shows the breakdown by type and seriousness, and Table 4 lists the main ones.

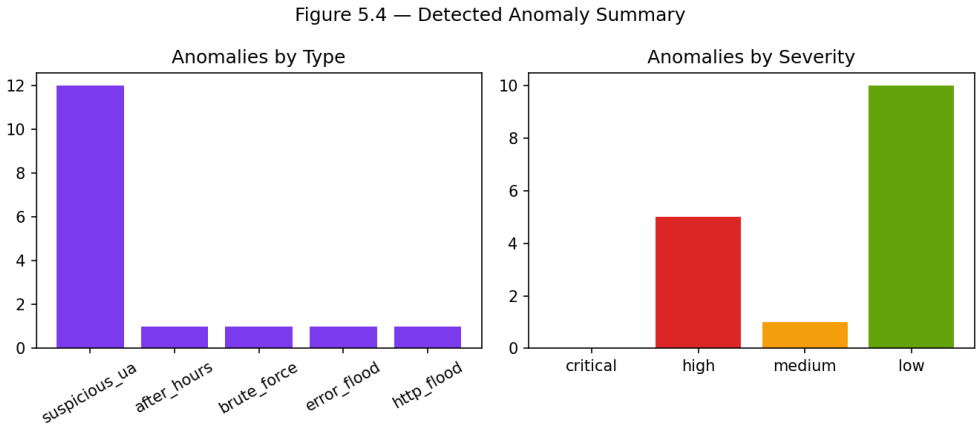

Figure 5 — Anomalies Found, By Type and Seriousness

Type	Seriousness	Visitor (IP)	Count	What It Means
Repeated failed logins	High	198.51.100.23	12	12 failed logins in 5 minutes
Too many 404 errors	High	198.51.100.55	25	25 page-not-found errors in 1 minute
Suspicious software	High	198.51.100.55	25	Used known scanner-tool name (Nmap)

Suspicious software	High	198.51.100.77	150	Used automated-script-style software
Too many requests	Medium	198.51.100.77	150	150 requests in 1 minute
Odd-hour access	High	203.0.113.99	6	Accessed /admin outside business hours
Suspicious software (x9)	Low	Various	1 each	One-off scanner-style visits

Table 4 — Sample Detected Anomalies

5.6 What This Tells Us

Every type of suspicious activity the project was built to catch, actually showed up correctly in the results — nothing was missed. The seriousness levels also made sense: the biggest, clearest attacks (the request flood, the scanning burst, the login attempts) were marked as high, while one-off, minor cases were marked low. This suggests the simple "compare count to a limit" approach works reasonably well, even without anything fancy like machine learning.

CHAPTER VI: DISCUSSION AND RESULTS

6.1 Testing the Website Itself

Apart from checking the detection logic, we also manually tested the website's features together to make sure they behave as expected. Table 5 lists what we tested.

What We Tested	What Should Happen	What Happened
Uploading a web log file	File is read and saved correctly	Worked — 803 lines read, 0 skipped
Uploading a login log file	File is read and saved correctly	Worked — 23 lines read, 0 skipped
Uploading a wrong file type	Should show an error message	Worked — clear error shown
Running the detection checks	Should find and list anomalies	Worked — 15 anomalies found
Running checks with no data	Should show a friendly error	Worked — “no logs” message shown
Clearing all data (reset)	Everything should be wiped	Worked — counts went back to zero

Table 5 — Test Cases and Observed Results

6.2 What We Learned / Discussion

Splitting the project into separate parts (reading logs, saving data, checking for problems, showing the dashboard) made it much easier to test and fix each part on its own, without breaking the others. This turned out to be a good decision, especially while debugging.

One honest limitation we noticed: the “suspicious software” check will sometimes flag things that aren't actually attacks — for example, a normal automated script also gets flagged, just at a low seriousness level. We chose to keep it this way on purpose, because it's safer to flag a few harmless things than to miss a real one. But it does mean a person still needs to glance at the low-priority items rather than trust the system blindly.

Comparing this back to our objectives in Chapter II, we were able to meet all of them: reading multiple log formats, storing the data, running five detection checks, giving each a seriousness level, building a dashboard, and testing it all on a real (sample) dataset.

CHAPTER VII: CONCLUSION

This project set out to build a simple tool that could read server log files and automatically point out signs of trouble, without needing expensive or complicated security software. The result, LogSentinel, does exactly that: it reads web server and login logs, stores them, checks for five common warning signs (repeated failed logins, error floods, request floods, suspicious visitor software, and odd-hour access), and shows the results on an easy-to-read dashboard.

When tested on a sample log file, the project correctly found every type of suspicious activity it was designed to catch, and correctly ranked how serious each one was. This shows that even a small, student-built project using simple, free tools (Python, Flask, pandas, SQLite, Chart.js) can do a useful, basic version of the job that expensive security software normally does.

CHAPTER VIII: RECOMMENDATIONS AND SUGGESTIONS

Based on what we learned while building and testing this project as a team, here are some ways it could be improved in the future:

17. Make the checks run only on new log entries instead of re-checking everything each time, so it can handle much larger, busier servers.
18. Add a login system for the dashboard itself, since right now anyone who can open the website can see everything.
19. Send automatic alerts (like an email) when something “high” or “critical” is found, instead of requiring someone to keep checking the dashboard.
20. Support more log formats, like logs from cloud servers or app-specific logs.
21. Let the user mark certain tools as “trusted” so known, harmless automated scripts stop showing up as low-level warnings.

REFERENCES / BIBLIOGRAPHY

1. SANS Institute — Log Management: Best Practices for Security and Compliance.

2. Apache Software Foundation — Apache HTTP Server Log Files Documentation.

3. NGINX, Inc. — NGINX Logging Module Documentation.

4. Pandas Documentation — pandas.pydata.org

5. Flask Documentation — flask.palletsprojects.com

6. Chart.js Documentation — chartjs.org

7. SQLite Documentation — sqlite.org

APPENDICES

Appendix A: Main Website Features

Feature	What It Does
Upload log file	Reads and stores a log file the user uploads
Load sample data	Loads the built-in demo log files for testing
Run detection	Checks stored data for the five types of suspicious activity
Reset	Clears all stored data
Dashboard charts	Shows request counts, status codes, top visitors, and anomalies

Appendix B: Sample Raw Log Lines

A typical web server log line:

203.0.113.99 - - [01/Jul/2026:02:10:00 +0000] "GET /admin HTTP/1.1" 401 512

A typical login (syslog) log line:

Jul 1 03:00:00 webhost sshd[1000]: Failed password for invalid user git from 192.0.2.88 port 40000 ssh2